Notes: Decision-focused learning for battery control

1 Problem statement

Consider a time-slotted system with a horizon T. We have an energy storage with capacity B. In the beginning of each time slot, we observe a demand d_t that must be fulfilled. This demand can be supplied by duel sources: (i) the discharged energy from the energy storage or the purchased energy from real-time energy market. Let p_t denote the energy price at slot t. Our problem is to decide the amount x_t of purchased energy from the market such that the total energy cost is minimized while all demands are satisfied. Thus, the problem can be formulated as

$$\min_{x_t \geq 0, s_t \geq 0} \quad \sum_{t \in [T]} p_t x_t \tag{1a}$$

$$\text{s.t.} \quad s_t = s_{t-1} + x_t - d_t, \forall t \in [T] \tag{1b}$$

$$x_t \geq d_t - s_{t-1}, \forall t \in [T], \tag{1c}$$

$$s_t \leq B, \forall t \in [T]. \tag{1d}$$

2 Task 1: Optimization without uncertainty

Given prices $\boldsymbol{p} := \{p_t\}_{t \in [T]}$ and demand $\boldsymbol{d} := \{d_t\}_{t \in [T]}$ over the time horizon, solve for the optimal charging decisions $\boldsymbol{x}^* = \{x_t^*\}_{t \in [T]}$ using cvxpy. Read Chapter 1 of the convex optimization book if you want to know more about optimization.

2.1 Numerical setup

We consider a time horizon of $T = 24$ time slots (to represent 24 hours in the day). The energy price p_t and demand d_t for each time slot are given by

$$\begin{aligned} \boldsymbol{p} = [&5, 4, 3, 2, 2, 3, 4, 6, 8, 10, 9, 7, \\ &6, 5, 4, 3, 2, 3, 5, 7, 9, 8, 6, 5], \end{aligned} \tag{2}$$

and

$$\begin{aligned} \boldsymbol{d} = [&4, 5, 4, 3, 3, 4, 6, 8, 10, 9, 8, 7, \\ &6, 5, 5, 6, 7, 8, 10, 11, 9, 8, 6, 5]. \end{aligned} \tag{3}$$

The storage capacity is set to $B = 20$. We assume that the initial storage level is zero, i.e., $s_0 = 0$. The optimization problem is implemented in CVXPY as follows:

```
import cvxpy as cp
import numpy as np

# Given data
T = 24

p = np.array([
    5, 4, 3, 2, 2, 3,
    4, 6, 8, 10, 9, 7,
    6, 5, 4, 3, 2, 3,
    5, 7, 9, 8, 6, 5
])

d = np.array([
```

```
    4, 5, 4, 3, 3, 4,
    6, 8, 10, 9, 8, 7,
    6, 5, 5, 6, 7, 8,
    10, 11, 9, 8, 6, 5
])

B = 20

# Decision variables
x = cp.Variable(T, nonneg=True)
s = cp.Variable(T, nonneg=True)

# Defining min(sum(p*x)) over time
objective = cp.Minimize(p @ x)

# Setting up Constraints
constraints = []

for t in range(T):
    if t == 0:
        s_prev = 0
    else:
        s_prev = s[t-1]

    constraints.append(
        s[t] == s_prev + x[t] - d[t]
    )

    constraints.append(
        x[t] >= d[t] - s_prev
    )

    constraints.append(
        s[t] <= B
    )

# Solve optimization problem
problem = cp.Problem(objective, constraints)
problem.solve()

print("Optimal cost:", problem.value)
print("Optimal purchases x*:", x.value)
print("Storage levels s*:", s.value)
```

2.2 Optimal solution

The optimal objective value obtained by CVXPY is

$$\boxed{637}. \tag{4}$$

Thus, the minimum cost of purchasing enough energy to satisfy all 24 periods of demand, while respecting the storage capacity constraint $B = 20$, is approximately 637.

The optimal market purchases are

$$\boldsymbol{x}^* \approx [4,\ 5,\ 4,\ 10.2187,\ 15.7813,\ 4,\ 6,\ 8, \tag{5}$$

$$7,\ 0,\ 0,\ 7,\ 6,\ 5,\ 5,\ 6, \tag{6}$$

$$27,\ 8,\ 10,\ 8,\ 0,\ 0,\ 6,\ 5]. \tag{7}$$

The corresponding storage levels are

$$\boldsymbol{s}^* \approx [0,\ 0,\ 0,\ 7.2187,\ 20,\ 20,\ 20,\ 20, \tag{8}$$
$$17,\ 8,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0, \tag{9}$$
$$20,\ 20,\ 20,\ 17,\ 8,\ 0,\ 0,\ 0]. \tag{10}$$

2.3 Interpretation

The optimal solution exploits the changes in the price of energy across time. When the price is low, the optimizer purchases additional energy beyond the immediate demand and stores the surplus. When the price of energy is high, then the stored energy is discharged to satisfy demand instead of purchasing all of the required energy from the market.

For example, at time slots 4 and 5, the prices are both 2. The optimizer purchases additional energy and completly fills the battery. During high-price periods, such as 10 and 11, where the price is 10 and 9 dollars respectively, the battery is discharged, resulting in zero market purchases at high price points. Optimization is effectively finding the cheapest way to satisfy demand.

3 Task 1.5: Real data and battery value

We solve the battery control problem using real demand and price data over one day.

- **Demand data:** Use the demand from generative AI workload in data center. Try this dataset. Find the whole facility profiles that can be understood as the power profile of a data center. The current granularity is 1 minute. Convert it to one hour.
- **Electricity price data:** Use the ERCOT day-ahead prices. See the dataset. Use the hourly day-ahead prices for a particular location.

Let $\xi_i = (\boldsymbol{p}^i, \boldsymbol{d}^i)$ denote the demand and price profiles for day i.

- For given ξ_i and battery capacity B, solve the optimization problem to evaluate the cost. Let $Q(B, \xi_i)$ denote the optimal cost under profile ξ_i and battery capacity B.
- Fix B, and solve the optimization problem N times, each with profile $\xi_i, i = 1, \ldots, N$. The sample average cost of using battery control over N days is $C_N(B) = \frac{1}{N} \sum_{i \in [N]} Q(B, \xi_i)$.
- Solve above problem for different battery capacity B, over the same N-day profiles. Draw a figure to show how costs will vary as battery capacity increases.

3.1 Data pre-processing

Demand Data

Firstly, we take the demand profile from the generative AI workload. We will use the file `inference_1MW_283nodes_60u_power.csv`, which is a simulated 1 MW inference data center. The file records the facility power $P(\tau)$ in watts once every minute, from the 1st of January 2018 00:00 to 31 December 2018 23:59 (525,600 samples).

Power is a *rate* of energy use (1 W = 1 J/s), whereas prices are charged per unit of *energy* ($/MWh). To obtain the energy used in each hour, we multiply each reading by the length of its interval, $\Delta\tau = 1/60$ hour, and sum over the 60 minutes in the hour. For day i and hour t, the demand is

$$d_t^i = 10^{-6} \sum_{m=1}^{60} P(\tau_{m,t}^i)\,\Delta\tau = 10^{-6} \cdot \frac{1}{60} \sum_{m=1}^{60} P(\tau_{m,t}^i) \quad \text{[MWh]}, \tag{11}$$

where $\tau_{t,m}^i$ is minute m of hour t on day i, and the factor 10^{-6} converts Wh to MWh. That is, the hourly energy demand is the mean power over the hour multiplied by one hour. Since each time slot is exactly one hour long, d_t^i in MWh is numerically equal to the average power in MW over that hour.

We then split these hourly series by calendar date into vectors $\boldsymbol{d}^i = \{d_t^i\}_{t\in[T]}$ with $T = 24$, which gives one demand profile for each of the 365 days of the year.

Price data

The price data is taken from the ERCOT day-ahead market hourly locational marginal prices (LMPs) in \$/MWh. Each file corresponds to one day, from September 8th till September 16th 2026, and lists the price for every hour at a different bus. For each file, we

1. keep only the rows for the bus `ADK_V_C`;
2. convert the hour-ending label (1-24) into time slot indexes (0-23) and check that all 24 hours are present;
3. store the resulting vector $\boldsymbol{p}^i = \{p_t^i\}_{t\in[T]}$ under the file's delivery date, with $T = 24$.

Pairing Price and Demand data

Now, we proceed to pair each price profile with each demand profile. Since our demand data are from 2018, while our price data are from 2026, we decided to pair the data by month and day instead. Furthermore, we have a small sample size of data (only 9 days in the month of September). Therefore, we only have a dictionary of 9 entries, where each entry is composed of an array of 24 entries of demand and an array of 24 entries of prices (1 for each hour). Below is a summary table of each day's mean price, peak price (in dollars), mean demand, and their respective peak demand (in megawatts).

Month-Day	Mean Price	Peak Price	Mean Demand	Peak Demand
09-08	42.499583	84.59	0.449850	0.539001
09-09	46.547917	103.14	0.349504	0.416922
09-10	46.142083	96.01	0.596762	0.801213
09-11	34.043750	53.47	0.604634	0.801175
09-12	31.139583	54.03	0.604813	0.801235
09-13	31.998333	56.52	0.597921	0.800905
09-14	31.965417	49.54	0.569909	0.776299
09-15	33.842083	60.09	0.441749	0.528621
09-16	42.720417	72.64	0.343240	0.408281

Table 1: Daily mean and peak electricity prices and demand.

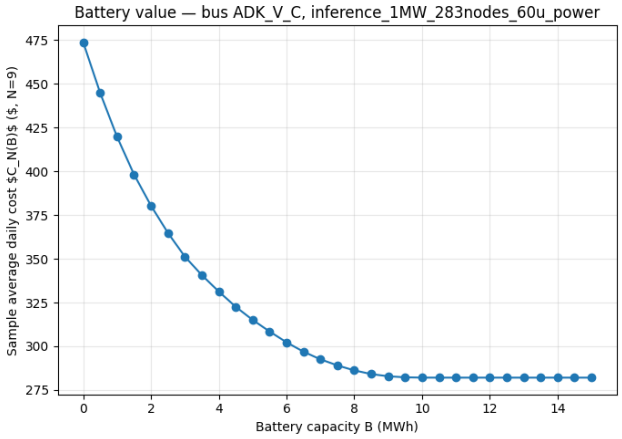


Figure 1: Cost vs Battery capacity for bus ADK

Battery Optimization Results

For each daily demand/price profile $\xi_i = (p_i, d_i)$, we solve the battery optimization problem to determine the minimum cost of meeting the hourly electricity demand given a battery capacity B. For each fixed battery capacity B, the optimization problem was solved independently for each of the N daily demand and price profiles. We then calculated the sample average daily cost as$[C_N(B) = \frac{1}{N}\sum_{i=1}^{N} Q(B, \xi_i),]$ where $Q(B, \xi_i)$ denotes the optimal electricity cost for profile ξ_i.

We evaluated battery capacities from 0 to 15 MWh in increments of 0.5 MWh. The average daily cost of the resulting sample decreases as the battery capacity increases from 473.76 without a battery to 282.09 at a capacity of 10 MWh. Beyond 10.5 MWh, the average cost remains constant at approximately 282.07 per day, indicating that additional battery capacity does not reduce the cost of the demand and price profiles considered.

4 Task 2: Learn-Then-Optimize

In this task, we assume that demand is given, while electricity prices are uncertain. To solve the optimization problem (1), we adopt a two-stage learn-then-optimize approach: first, we learn the electricity prices in the first stage, and then use the predicted prices to solve the optimization problem.

Sub-task 2.1: Price prediction. We should first note that, since the objective function is linear in the electricity prices and the feasible set does not depend on the prices, the expected cost optimization problem only depends on the conditional mean price in each time slot.

To start with, build a neural network model to predict the mean prices $\mu_t(z;\theta), t = 1, \ldots, T$, where z denotes the available features, such as historical prices and calendar information, and θ denotes the parameters of the neural network. The model can be trained by minimizing the mean squared error between the predicted mean prices and the observed prices.

After obtaining the predicted means, estimate the variance $\hat{\sigma}_t^2$ for each time slot based on the prediction residuals, i.e., $p_t - \mu_t(z;\theta)$. For simplicity, we can assume that the variance for each time slot is fixed across different days.

We then assume that the electricity price at time t follows a Gaussian distribution:

$$p_t \mid z \sim \mathcal{N}\left(\mu_t(z;\theta), \hat{\sigma}_t^2\right).$$

The prediction approach in Donti's paper is largely based on this idea of first learning the mean and then characterizing the prediction uncertainty. Note that their battery experiment uses a lognormal price model. You can use either a Gaussian model for simplicity or a lognormal model.

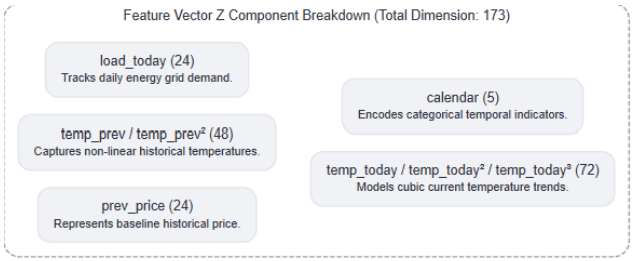


Figure 2: Component breakdown of Feature vector Z

Configuration

We will be using two files for our task: The demand file from the generative AI workload data center, as well as the price file used by Donti, which originates from PJM. We will also be using a lognormal price model, however the implementation covers the ability to also run the experiment with a Gaussian model. We will be conducting the experiment with a 70% training split to learn the neural network parameters, a 10% split for validating and tuning the model and detecting overfit, and 20% for testing. Furthermore, according to Donti, the price file contains numerous price spikes and outliers. To reduce the influence of extreme observations on our price prediction model and batter optimization, we decided to exclude prices outside the 0-500$/MWh range.

Data-Preprocessing

The shape of our price data is 52,560 rows x 4 columns, where each row corresponds to a date and hour and the columns correspond to the day-ahead price, the load forecast, or how much electricity is expected to be consumed, as well as the temperature outside. We transform our original data into 3 separate tables (one for prices, one for load forecasts, and the other for temperature), where the columns correspond to the hour of the day and the rows are the dates. We also fill any missing data with the value of the next hour. Afterwards, we filter price based off our outlier range, and transform all the prices into log(price). Generally, only 6 cells were removed, all for exceeding the limit of 500 $.

Now that we have filtered our dataset, and created our tables, we proceed to creating a machine-learning dataset, where every row is a day that contains 173 features used to predict a day's 24 electricity prices, defined by the following function:

$$f_\theta : \mathbb{R}^{173} \to \mathbb{R}^{24}$$

We can split the features into 7 categories, where we have calendar features, which tells us if its a weekend, holiday, daylight savings time, or the position of the day in the yearly cycle (in cosine and sine). Furthermore, we get yesterday's prices, the load demand for today's prices, the temperature of yesterday, and today's temperature.(Note: I am not sure how significant it is; but we may remove today's temperature from the features as this is something that we may not know and also need to predict.) We also include the squared temperature as well as the cubed temperature in our feature vector as electricity demand skyrockets when temperatures are extremely high or extremely low. Thus, we want our model to capture the non-linear relationship between price and temperature.

Finally, since every component in our new feature vector Z has different numerical sizes, we standardize everything to ensure that the optimization landscape is not poorly conditioned and that we can have faster and more stable training.

Training/Validation/Test setup

The dataset is divided chronologically into training, validation, and test sets, with 1528 days used for training, 218 days used for validation and 437 days used for testing. The chronological split ensures that future observations are not used in training the model. Additionally, the training set is

shuffled and divided into mini-batches. Without shuffling, we would get 64 consecutive winter days, a biased sample. Meanwhile, the validation and test sets are kept in chronological order so that predictions remain aligned with their corresponding dates and realized prices. The neural network is trained using MSE and the adam optimizer with a learning rate of 10^-3. During training, we monitor the validation MSE after each epoch, and the model parameters with the lowest validation MSE are saved. We also utilize ReLU, as we want to learn nonlinear relationships, since temperature -¿ electricity demand -¿ price is not necessarily linear. We also use batchNorm, which normalize the outputs of the 200 neurons using the current mini-batch. Afterwards, BatchNorm shifts and scales that allow the neural network to undo or adjust the normalization if it helps the model learn better by adding a bias value and changing the spread of the data. Finally, we also use another technique called drouput, which randomly switches off 20% of neurons to reduce overfitting, to force the NN to not be dependent on particular neurons. Training uses early stopping with a patience of 20 epochs to prevent overfitting. The test set remains untouched and is only used for the final evaluation of the model.

Training results

According to the training results, the neural network learns useful patterns in electricity prices but begins to overfit after 12 epochs. The training MSE falls substantially from 0.09594 at epoch 0 to 0.01316 at epoch 10, showing that the model learns the relationship between input features and the 24 hourly log-prices. The MSE validation decreases as well from 0.02866 to 0.01894, indicating that this improvement is geniune and the model is starting to generalize to unseen days. However, further epochs show that the training MSE continues to lower, but we only see slight variations in validation MSE, showcasing classic overfitting.

Epoch	Training MSE	Validation MSE
0	0.09594	0.02866
10	0.01316	0.01894
20	0.01053	0.01814
30	0.00929	0.01916

The best validation performance was achieved at epoch 12, with

$$\boxed{\text{Validation MSE} = 0.01697}$$

in log-price units. In RMSE, that corresponds to a validation error of

$$\text{RMSE} = \sqrt{0.01697} \approx 0.130.$$

in log-price space. We can use this to calculate a multiplicative difference of approximately 14%, using:

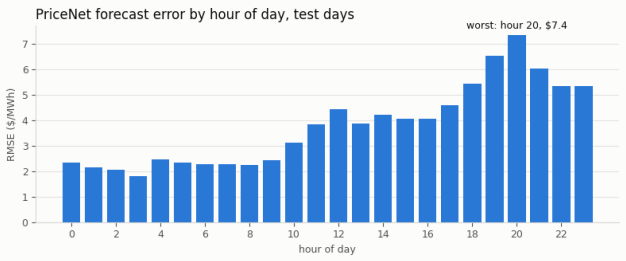
$$e^{0.13} \approx 1.139,$$

Test results

When testing our model, we do not want to be checking if its able to just predict the price well, but also if it is better at calculating some hours better than others. We do this by checking by checking the RMSE, the Mean Absolute Error, and the Mean Absolute Percentage Error We include MAE and MAPE since it demonstrates the magnitude of the error. We also create two baselines: persistence and hour-of-day mean. The persistence forecast is an assumption that tomorrow's prices will be the same as yesterday's prices, while the hour-of-day mean forecast calculates the average price for each hour using training data only, and uses those 24 values for every test day.

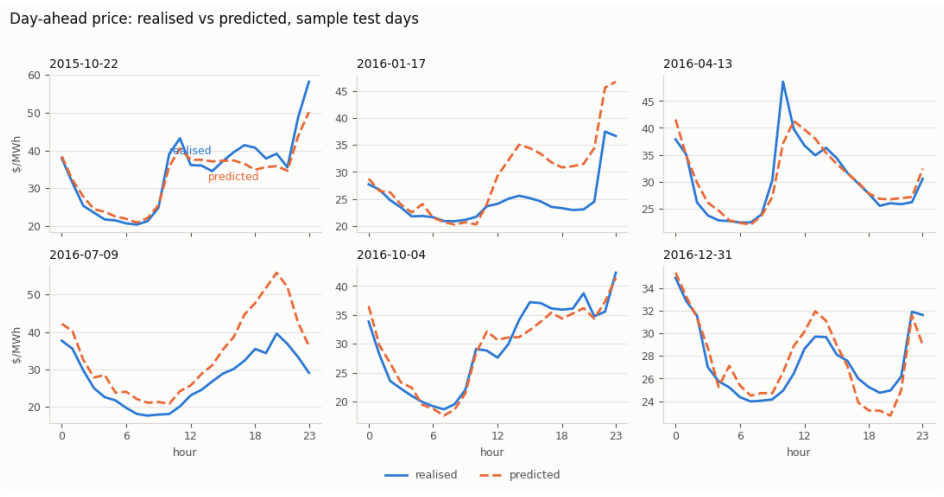
Table 2: Test-set price forecasting performance. Lower values indicate better performance.

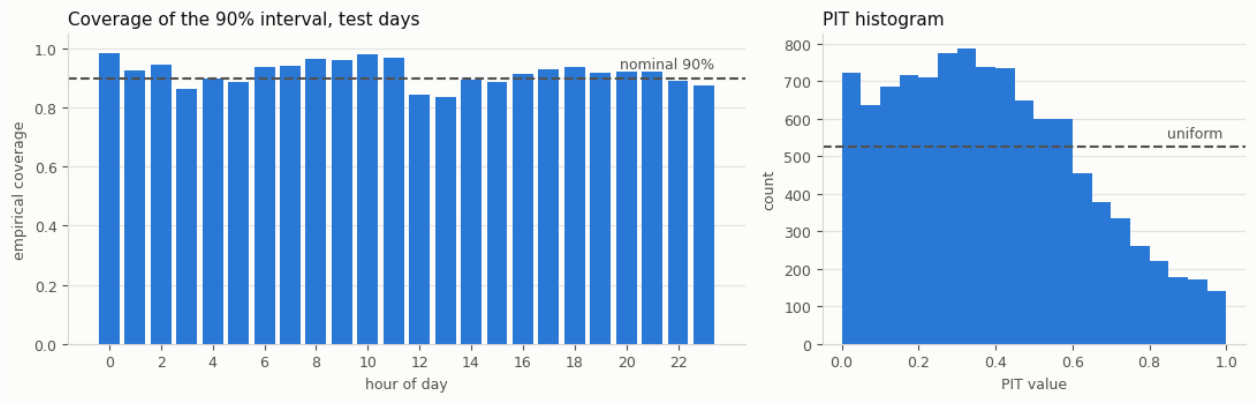
Method	MSE (log)	RMSE (log)	RMSE ($/MWh)	MAE ($/MWh)	MAPE (%)
Neural Network	**0.0135**	**0.1162**	**4.0927**	**2.5241**	**9.0065**
Persistence	0.0345	0.1856	6.2586	3.8235	13.1237
Hour mean	0.1813	0.4258	12.8061	11.3064	47.9033



Given the results above, we notice that our neural network clearly improves upon both the simple baselines in price-forecasting accuracy. The comparison tells us that persistence was a strong baseline, as yesterday's prices contain useful information about today's prices. However, our neural network was not only trained on yesterday's prices but other additional information, such as temperature, and load-demand. Thus, the results suggest that this additional information provides useful information. We notice in the forecast error by hour of day graph that our model is great at predicting prices during off-peak hours, but as demand increases throughout the day, the forecast errors increase upwards, suggesting that the model has a hard time at predicting the volatility in price associated with peak hours. We notice that the model overestimates whenever demand is moderate, as shown in the day-ahead price graph, on the days 2016-01-17 and 2016-07-09. However, it also severely underestimates price spikes later in the day, like on 2015-01-22.

However, to account for the uncertainty in predicted electricity prices, a probability distribution was estimated for each hourly price rather than relying solely on a point prediction. The neural network provides the condition mean (μ_t), while the variance ($\hat{\sigma}_t^2$) was estimated by getting the model's residuals on the validation set. Under the lognormal assumption, $(\log(p_t) \mid z \sim \mathcal{N}(\mu_t, \hat{\sigma}_t^2))$, which gives the expected price as $(\mathbb{E}[p_t \mid z] = \exp(\mu_t + \hat{\sigma}_t^2/2))$. This distribution is important as the battery optimization problem requires expected electricity prices and allows for uncertainty of the forecasts to be quantified. This The estimated standard deviations ranged from 0.065 at hour 3 to 0.227 at hour 10. This demonstrates greater uncertainty in the morning hours. The mean validation residual was -0.0216, which indicates a small upward bias when predicted prices, showcasing that





our model was generally predicting higher prices that normal. We asked our model to output a range of prices that it expected the real price to fall into 90% of the time. However, the real price fell in about 91.8% of the time, showcasing that the model was slightly over-confident in its uncertainty. On the other hand, the PIT mean was 0.389, below the ideal value of 0.5, showcasing the systematic upward forecast bias. Thus, our model tends to predict prices that are slightly higher than what actually happened. In summary, while the prediction intervals achieved close to the desired overall coverage, the PIT analysis shows a small centering bias.

Sub-task 2.2: Optimization using predicted prices. Given the predicted price distribution in Sub-task 2.1, solve the battery optimization problem and compare the resulting solution with the hindsight solution, where the optimization is performed using the true realized prices.

Set-up

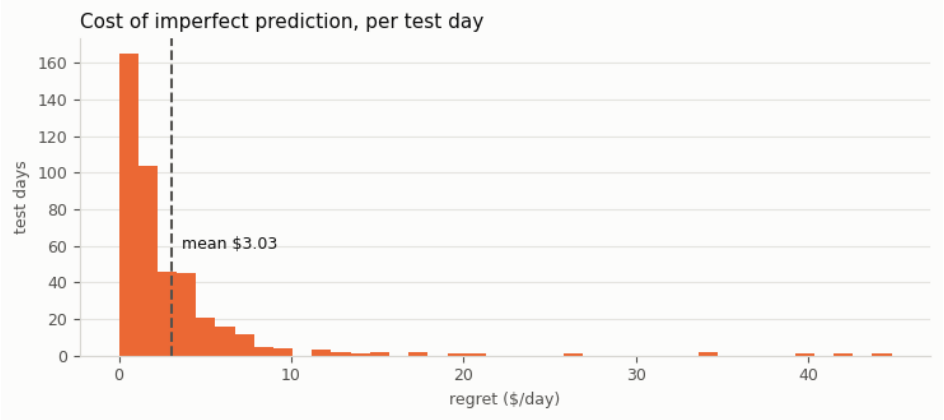
For our demand data, we will be using the task 1.5 profile (1 MW inference data center, hourly MWh), and match each test day by (month, day), as demand year and price year differs. Then we set up the battery optimization problem, where price is uncertain and the only way our optimization problem knows about it is through the predicted distribution made by our neural network, where we look to optimize:

$$
\begin{aligned}
& \min_{x} \sum_{t=0}^{23} p_t x_t \\
\text{subject to:} \quad & s_0 = x_0 - d_0 \\
& s_t = s_{t-1} + x_t - d_t, \quad \forall t \in \{1, \ldots, 23\} \\
& 0 \leq s_t \leq B, \quad \forall t \in \{0, \ldots, 23\} \\
& x_t \geq 0, \quad \forall t \in \{0, \ldots, 23\}
\end{aligned}
$$

Results

To evaluate our optimization, we use the cost of electricity if there was no battery price, the cost of electricity using the electricity prices predicted from our neural network, and the cost of electricity if we had 100% perfect price forecast (price hindsight). We utilize regret, which is the difference in cost of electricity with predicted prices and cost of electricity with hindsight to calculate the performance of our model, with a lower regret meaning that our optimized result given predicted prices is closer to the optimal solution.

In addition, we utilize the previous baselines of persistence and hourly-mean to compare with the performance of our neural network. Finally, we have the captured column, which represents the percentage of savings obtained by our method relative to having no battery or:



$$\text{Battery Value} = \frac{\text{NoBat Cost} - \text{Model Cost}}{\text{NoBat Cost} - \text{Hindsight Cost}}$$

We first start by testing a battery capacity of 5.0 MWh on 436 test days:

Table 3: Battery optimization results for $B = 5.0$ MWh over 436 test days.

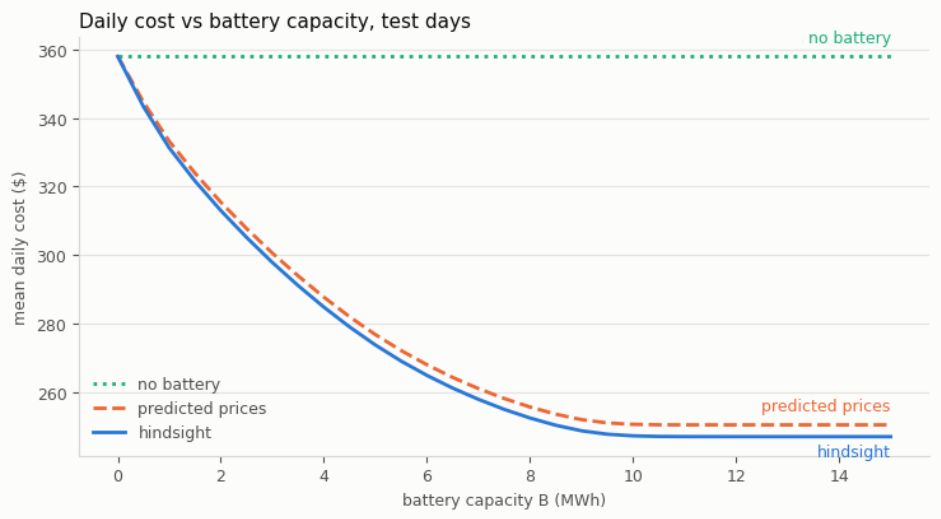
Method	**Mean Cost ($/day)**	**Mean Regret ($/day)**	**Captured**
No battery	357.923	–	0.000
Neural Network	276.778	3.030	0.964
Persistence	278.038	4.290	0.949
Hour mean	286.281	12.532	0.851
Hindsight	273.748	0.000	1.000

Based on the table, we can see that our neural network does a good job at predicting prices, as it manages to capture 96.4% of available savings. (Note: Results in this table may not lead to this conclusion. The table can show that the learn-then-optimize approach can attain a good decision on average; if we want to comment on the quality of prediction, need to refer to table 2.) Furthermore, the average regret is on 3.03$, which signals that our model is allowing us to get very close to the optimal solution. Another thing to note is the performance of persistence, which does not lag too far behind from our neural network's performance. This tells us that persistence, despite being a zero-parameter model, is a powerful operational benchmark and backup option.

However, if we look into the error distribution of our neural network, we notice that it is heavily skewed with a long right tail. At the 50th percentile, our neural network looses 1.63$ or less relative to perfect hindsight, showcasing that our model performs well on well-behaved days. However, at the 90th percentile, our regret increases to 6.29$. The maximum regret is 44.83$. This tells us that on days where high-volatility is apparent, our model executed a charging/discharging strategy at wrong times, leading to significant financial loss in comparison to perfect hindsight.

Now, we decided to run the same thing over multiple batter capacities, ranging from 0 - 30 MWh.

According to the figure, we experience strong diminished returns after a battery capacity of 10, both with predicted prices and actual prices. Furthermore, it seems that we have a high capture rate, as we consistently capture 95% to 97% of the maximum theoretical savings achievable by a all knowing oracle. Furthermore, absolute regret seems to remain stable ranging only between 2.44$ and 3.43$. Finally, both optimizations with predicted and actual prices agree that the sweet-spot for battery capacity is between 5 - 10 MWh, while any capacity after 12 does not yield any improvements.



4.1 Analysis and discussions on current results

First, from the prediction results (e.g., Table 2 and Figure **??**), the prediction does not seem to be perfect; however, the decisions from the downstream optimization seems to work well. This means the optimizer does not care that much about the prediction accuracy uniformly. I may only care whether the prediction correctly identifies when electricity is relatively cheap and when it is relatively expensive.

Second, the interesting case is to observe that some days may have very large regret. Thus it deserves to investigate why this happens. We may investigate the following aspects:

- For each test day i, calculate the regret (say, $R_i = C_i^{NN} - C_i^{off}$) and report the regrets' mean, P_{90}, P_{95} and max.
- examine the worst 5-10 days; compare the predicted prices and true prices, decisions, and battery level trajectory.
- compare prediction error and regret. Plot a figure with x-axis as daily price RMSE and y-axis as daily regret to see the relationship between the prediction accuracy and decision accuracy.

This relates to control the risk in decision-focused learning. Please refer to this Caltech paper for reference [2].

Preparation for analysis To prepare for analysis, we proceed to take all the purchases and battery prices to, and the optimal purchases and battery levels that the hindsight solution leads to, and calculated the following metrics for analysis.

- Regret: the cost of electricity in a day, given predicted prices - the optimal cost of electricity
- saving hindsight: the cost of no battery - the optimal cost
- Lost share: regret / saving hindsight, which leads to the fraction of the possible saving that the forecast threw away
- RMSE: daily RMSE in $/MWh
- bias: the average error, or tendency of our model of over-predicting or under-predicting prices relative to actual values
- Shape RMSE: evaluates the forecast error left by removing the constant offset, or bias.

- Rank Correlation: A spearman correlation between predicted and true prices, which allows us to determine whether the predicted prices were in the right order, i.e. if the both variables consistently increase or decreases together.
- Trough miss: The true price at the hour the neural network thinks its cheapest minus the true cheapest price, to see how much it costs per MWh to charge at wrong hour.
- Hour minimums: To determine which hour has the global minimum of the day for both predicted prices and true prices (to see how far off we are)
- True spread: To see how volatile the day was in terms of price, by taking the peak price of the day - the smallest price of the day.
- the L_1 difference: The total physical displacement between decisions in a day.

Regret Analysis Judging on the table, it seems that on average, our learn + opimize approach

Table 4: Regret per Test Day ($/day) across $N = 436$ evaluation days for a battery storage capacity of $B = 5.0$ MWh.

Model	Mean	P50	P90	P95	P99	Max	Worst 5% Share	Worst 10% Shar
Neural Network	3.030	1.630	6.288	8.956	31.477	44.825	0.331	0.451
Persistence	4.290	1.352	11.234	18.591	36.253	98.029	0.385	0.561
Hour Mean	12.532	8.085	28.953	40.959	73.772	88.224	0.230	0.363

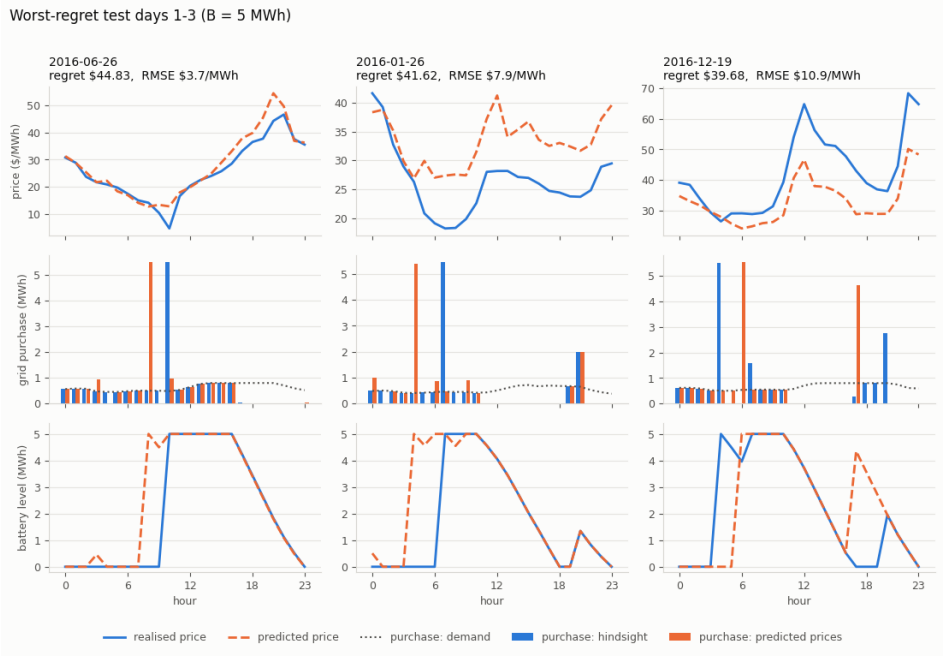
did a fine job in predicting prices, considering that the mean regret is the lowest. However, what is extremely interesting is that at the median, it gets beaten by persistence, showcasing that on typical calm days, the assumption of today's price profile looks like yesterday's leads to better decision making. However, as we continue towards the tail, we notice that the neural network, performs better than the other baselines, but starts becoming more inaccurate, with a big failure at the 99th percentile (yet a 8.9$ regret at the 95th percentile is impressive, considering the other options falter). If we take a look at the 5% share, which corresponds to how much of the 436-day regret falls within 5% of the worst test days. As we can see, that number is 33.1%. At 10%, that number jumps to 45.1%. This demonstrates that we should take a closer look at the tail examples to try and see ways to improve performance, as they account for a huge portion of total financial regret, and is a good starting point towards improving the model.

Worst-days analysis

Table 5: Worst 10 Days by Regret ($B = 5.0$ MWh)

Date	Regret	saving_hind	lost_share	rmse	bias	shape_rmse	rank_corr	trough_miss	hour_true_min	hour_pred_min	true_spread	x_l1_diff
2016-06-26	44.83	170.99	0.26	3.68	2.03	3.07	0.99	9.45	10	8	41.91	10.97
2016-01-26	41.62	46.91	0.89	7.88	6.89	3.83	0.77	8.09	7	4	23.46	12.77
2016-12-19	39.68	175.11	0.23	10.88	−9.11	5.94	0.90	2.64	4	6	41.94	20.78
2016-02-15	34.40	64.13	0.54	9.39	6.98	6.29	0.71	2.64	9	20	29.77	13.23
2016-06-19	34.21	164.68	0.21	6.86	4.89	4.81	0.99	7.13	10	8	41.74	11.76
2016-02-14	26.41	57.69	0.46	16.68	13.37	9.98	0.68	0.00	20	20	34.74	13.74
2016-06-13	20.70	128.17	0.16	2.48	1.62	1.88	0.98	3.57	7	8	31.46	11.90
2016-01-18	19.19	123.11	0.16	11.45	8.44	7.74	0.88	2.13	6	3	41.59	14.30
2016-01-23	17.48	63.52	0.28	2.34	−0.43	2.31	0.87	3.11	4	11	17.04	13.89
2016-07-10	17.29	102.93	0.17	5.10	4.44	2.52	0.98	3.11	10	8	23.69	11.30

To do this, we decided to analyze the top 10 worst days by regret, to get a sense of what went wrong, and if there is a common point among these examples.



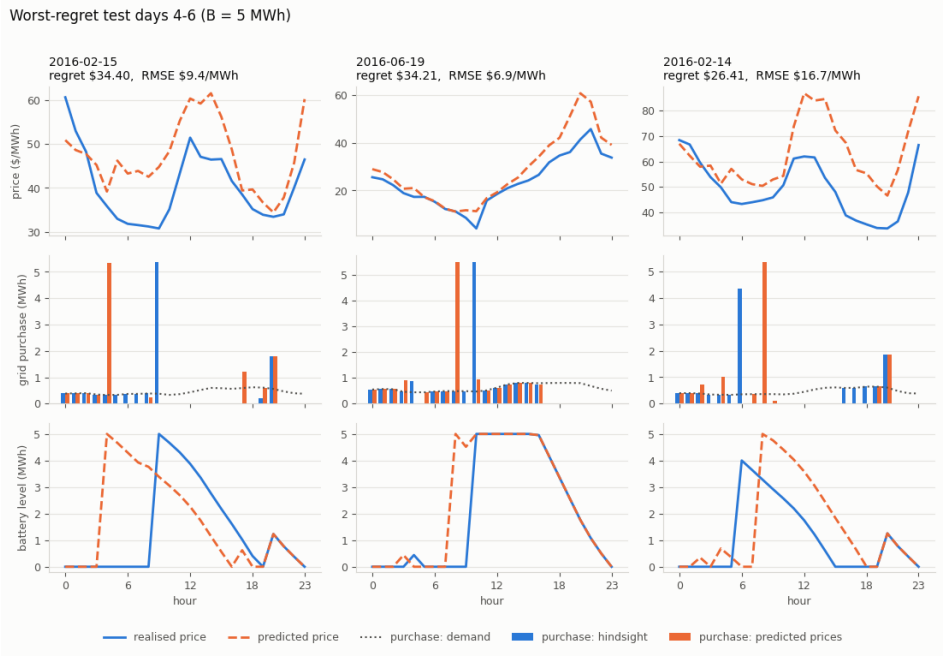
Firstly, judging by table 5, we can see right away that a low RMSE does not guarantee a low regret, as our worst day as only a RMSE of 3.68$/MWh, and a perfect rank correlation of 0.99. However, what is worth noting is the mismatches in the predicted minimum hour and the true minimum hour. It seems that the failure to predict the true minimum price hour is causing high trough misses, driving bulk purchasing errors. This is shown in the L_1 difference too, where during high-regret days, some reach an average of 13MWh, which means that we bought roughly 6.5MWh at the wrong time, or one entire battery capacity. This mismatch in minimum prices is shown across the charts below, where the model gets the minimum prices of the day wrong. If we take a look at the worst-regret days figures, we can classify each worst day into two buckets: summertime, and winter time, which lead to high regrets for different reasons. For example, days like 2016-06-19, 2016-06-26, have narrow price-trough windows. However, for winter days, the failure is caused by a failure across the whole shape, as we see many of the predicted prices are consistently higher than the true price or lower.

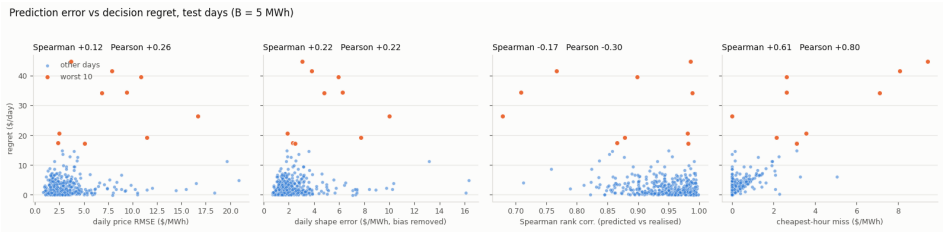
Prediction error vs Regret The original idea was to see the relationship between prediction accuracy and decision accuracy by plotting daily price RMSE vs daily regret. However, as the figure below shows, a low RMSE does not correspond to a low regret. We notice that our ten worst days (represented by orange dots) have a wide range, with the worst day having a RMSE of just 3.68. However, what stood out was the trough miss (cheapest hour miss), which had a high positive correlation with regret. Since the price penalty of missing the true minimum-price hour increases, decision regret rises proportionally.

Takeaways + discussion about the Conformal risk training

Based on what we have explored so far, it is safe to say that the weakness of the learn-and-optimize approach is not the average performance but the tail, where 10% of its test days account for 45% of the total regret, showing that there is a fundamental problem with how the network is trained. We trained on RMSE as a loss function, where it weights every hour of every day equally, whereas downstream decisions depend on where the extremes of the daily price curve fall. For example, a forecast that is 5$/MWh too high in every hour but identifies correctly the day's minimum will yield the optimal schedule and zero regret, since a constant shift in prices does not matter. Conversely, a forecast that is precise everywhere except on the day's minimum can be extremely costly and impact our decision and costs negatively.

I think that decision-focused learning [1] helps us by training the network on the realized cost of





its decisions instead of some statistical measure like price error. By propagating gradients through a differentiable optimization, we can improve performance. However, I think that this would minimize the average cost, but not deal with our specific issue: controlling rare, volatile days. Conformal risk training extends our decision-focused learning idea by not focusing on expected loss but rather a tail risk measure of the decision loss, like CVaR or the average loss over the worst α fraction of days, and combines end-to-end training with conformal risk control to bound this risk on unseen days. In our setting, setting α to 0.05 or 0.1 would correspond to the worst 5% or 10% of days we saw in our Table 4. By training against this objective, we would be able to concetrate the gradient on days where forecasts misplace the price trough (hour of the minimum price of the day), where our regret originates from, while leaving the typical days alone.

I highly suggest implementing task 3, then proceeding immediately with evaluating CRT as a way to reduce regret in the tail while preserving the low median.

5 Task 3: Decision-focused learning

From what we observed in last section, we may change our question to: Can decision-focused training preserve the already-good performance on ordinary days while reducing the relatively rare but expensive decision errors?

This part is to follow the DFL approach in the battery control problem in paper [1]. One technical issue here is that our downstream optimization is now a linear programming, which is hard to differentiate over. So we may consider the objective of our battery control problem has a

strongly convex regulator in the form of

$$\min_{x} \sum_{t=0}^{23} p_t x_t + \frac{\epsilon}{2}||\boldsymbol{x}||_2^2 \tag{12a}$$

$$\text{subject to:} \quad s_0 = x_0 - d_0 \tag{12b}$$

$$s_t = s_{t-1} + x_t - d_t, \quad \forall t \in \{1, \ldots, 23\} \tag{12c}$$

$$0 \leq s_t \leq B, \quad \forall t \in \{0, \ldots, 23\} \tag{12d}$$

$$x_t \geq 0, \quad \forall t \in \{0, \ldots, 23\} \tag{12e}$$

where $\epsilon > 0$ is a small number.

Sub-task 3.1: Battery optimization layer. Use cvxpylayers to code the optimization problem (12) as a differentiable optimization layer. After this step, test two things:

- verify that cvxpylayers and cvxpy produce the same solution when solving the same regularized problem. Then, test different small values of ϵ and verify that the regularized problem gives a solution and realized operating cost close to those of the original LP.
- Next, verify that gradients can be propagated through the optimization layer. For example, given predicted prices $\hat{\boldsymbol{p}}$, obtain $\boldsymbol{x}^*(\hat{\boldsymbol{p}})$ from the optimization layer and define a scalar loss $L(\hat{\boldsymbol{p}}) = \boldsymbol{p}^\top \boldsymbol{x}^*(\hat{\boldsymbol{p}})$, where $\boldsymbol{p}$ denotes the actual realized prices. Use automatic differentiation to calculate $\partial L / \partial \hat{\boldsymbol{p}}$ and compare several components with finite difference approximations.

Sub-task 3.2: decision-focused learning. This step is to train the neural network by considering the cost of the optimization problem. In previous learn-to-optimize approach, the NN is trained using the loss $L_{LTO}(\theta) = \frac{1}{N} \sum_{i \in [N]} ||f_\theta(z_i) - \boldsymbol{p}_i||_2^2$. For DFL, we obtain the optimization decision using predicted prices $\hat{\boldsymbol{p}}_i = f_\theta(z_i)$ to solve the problem (12), and the solution is $\boldsymbol{x}_i^*(\theta)$. Then the decision loss is evaluated using the actual realized prices $\boldsymbol{p}_i$, and it is $L_{DFL}(\theta) = \frac{1}{N} \sum_{i \in [N]} \boldsymbol{p}_i^T \boldsymbol{x}_i^*(\theta)$. To train the NN, we need to obtain the gradient

$$\frac{dL}{d\theta} = \frac{\partial L}{\partial \boldsymbol{x}^*} \cdot \frac{\partial \boldsymbol{x}^*}{\partial \hat{\boldsymbol{p}}} \cdot \frac{\partial \hat{\boldsymbol{p}}}{\partial \theta} \tag{13}$$

cvxpylayers computes the differentiation through the optimization layer, while PyTorch handles the remaining differentiation through NN.

Battery Optimization Layer

Task 3 consists of training the price model on the decision it leads to. Therefore, it needs the gradient of the linear programming solution with respect to the predicted prices. However, a LP solution gives an undefined gradient with respect to our parameters since the objective function is linear, its gradient is constant everywhere, and the only thing we know is that the true minimum or maximum is at the boundaries of the objective function.

To fix this, we need to add a smoothing term to turn our objective function into one with soft curves. Therefore, we choose to add $\frac{\epsilon}{2}||\boldsymbol{x}||_2^2$
to make the objective function unique and smooth, so that cvxpylayers can differentiate it.

When comparing cvxpylayers and cvypx's solution when solving the same regularized problem, we see that cvxpylayers solves the same problem as cvypx on all 436 test days, with error terms of 1, 0.1, 0.01, and 0.001. Based on table 6, we see that cvxpylayers solved our problem with a maximum objective difference of about 2.2×10^{-6}. However, it seems that the battery schedules differ by at most 0.0073 MWh for an error term of 0.01. It makes sense that the schedules are different since the quadratic penalty rewards the optimizer when it picks a charging schedule that is smoother and more evenly distributed. The error term would just influence how much the penalty influences the solution.

Table 6: Validation of the differentiable layer against CVXPY over 436 test days, with battery capacity $B = 5.0$ MWh

ϵ	$\max \lvert x_{\text{layer}} - x_{\text{CVXPY}} \rvert$ (MWh)	Maximum objective difference ($)
1.000	5.94×10^{-4}	2.17×10^{-6}
0.100	1.55×10^{-3}	1.79×10^{-6}
0.010	7.25×10^{-3}	1.65×10^{-6}
0.001	2.39×10^{-3}	1.83×10^{-6}

Table 7: Regularized battery optimization results

ϵ	Gap (MWh)	Cost ($/day)	Gap ($/day)	Gap (%)	Hindsight (%)
1.0000	3.9322	277.9227	1.1449	0.4137	0.8145
0.1000	2.9392	276.6311	-0.1467	-0.0530	0.0641
0.0100	2.2764	276.7728	-0.0050	-0.0018	0.0012
0.0010	2.1539	276.7845	0.0067	0.0024	-0.0000
0.0001	0.9103	276.7801	0.0023	0.0008	-0.0000

If we look at table 7, we see that the schedule difference and cost penalty decreases as our weight for our penalty term decreases.

We see that the gaps are bigger between the optimal charging schedule and cost versus an error term of $\epsilon = 1$. The schedules themselves stay apart, even for tiny ϵ. This is explained once again by our regularised problem returning the evenly spread schedule versus the linear programming's one vertex charging schedule.

So why bother with high ϵ values, if they are proven to be less accurate than having no regularization terms? We want to utilize gradients to help our model adjust its price predictions in a direction that improves the battery's decisions. By knowing how much the final costs changes when the predicted hourly price changes, we can calculate the gradients that show which predictions hurt the decision and update the model to reduce the cost. However, the size of ϵ determines the utility of the gradient. A large ϵ allows the battery to smoothly adjust how much it charges per hour, creating a meaningful gradient. However, a small ϵ can create near-zero or enourmous gradients, as say if we take the decision to fully charge at an hour, and charge nothing at all the other hours, and slightly increase the predicted price of the charging hour, if the change results in no changes in the optimal schedule, we end up with a near-zero gradient. However, if the change forces a full switch to a full charge at another hour, we end up with a huge gradient. In table 8, we see that as ϵ decerases, the median largest gradient also decreases, and the days with gradients below 0.01 percent increases dramatically.

Table 8: Gradient magnitude statistics across 40 test days

ϵ	Median	90th pct.	Max	Days with $\lvert g \rvert < 0.01$ (%)
10.000	0.4741	0.7211	1.8356	0.0
1.000	1.2198	2.3540	4.1627	0.0
0.100	2.5001	12.1398	42.1654	20.0
0.010	0.0002	0.7998	53.4985	90.0
0.001	0.0002	0.0004	1165.1274	97.5

Based off table 8, it is clear that training needs gradients that exist on every day, so as a starting

point, we can experiment with an error term of around 0.1 to 1, which costs about 0.06 to 0.08 percent above the optimal cost in table 7.

Decision-focused learning

References

[1] Priya Donti, Brandon Amos, and J Zico Kolter. Task-based end-to-end model learning in stochastic optimization. *Advances in neural information processing systems*, 30, 2017.

[2] Christopher Yeh, Nicolas Christianson, Adam Wierman, and Yisong Yue. Conformal risk training: End-to-end optimization of conformal risk control. *Advances in Neural Information Processing Systems*, 38:70056–70092, 2026.